BharatBuy: Intelligent E-commerce Assistant

Table of Contents :

None

```
BharatBuy Documentation Structure :
├─ 1. Introduction
│  ├─ 1.2 Problem Definition
│  ├─ 1.3 Existing System & Its Drawbacks
│  ├─ 1.3 Existing System & Its Drawbacks
│  ├─ 1.4 Need for the New System
│  ├─ 1.5 Objectives of the Proposed System
│  └─ 1.6 Scope & Core Modules
├─ 2. Project Profile & Feasibility
│  ├─ 2.1 Project Profile (Team, Tools, Target Audience)
│  ├─ 2.2 Hardware and Software Requirements
│  ├─ 2.3 Feasibility Study (Technical, Operational, Economic)
│  └─ 2.4 Required Determinants
│  ├─ 2.5 Targeted Users
├─ 3. Assumptions and Constraints
│  ├─ 3.1 Assumptions
│  └─ 3.2 Constraints & Dependencies
├─ 4. System Analysis & Design
│  ├─ 4.1 Use Case Diagram & Descriptions
│  ├─ 4.2 System Flowchart / Activity Diagram
├─ 5. Database Design
│  ├─ 5.1 Entity-Relationship (ER) Diagram
│  └─ 5.2 Data Dictionary / Data Tables (Schema, Types,
Constraints)
├─ 6. System Evaluation
│  ├─ 6.1 Key Advantages of BharatBuy
│  └─ 6.2 Limitations of the System
├─ 7. Future Scope & Proposed Enhancements
├─ 8. Conclusion
└─ 9. Bibliography & References
```

1. Introduction

1.1 Project Overview (BharatBuy)

This document provides a comprehensive technical overview and project documentation for the BCA Semester III User Defined Project. It outlines the full lifecycle of the BharatBuy system, covering the architectural design, business requirements, data modeling, implementation strategies, and future scope. The objective is to present a transparent roadmap of how the system was conceptualized to address current market inefficiencies in the online retail sector.

A key differentiator of BharatBuy is its 'Value Score' algorithm, which serves as a core pillar of the system's intelligence by ranking products based on holistic quality signals rather than price alone.

The modern e-commerce landscape in India has witnessed explosive growth, yet consumers are increasingly burdened by fragmented marketplaces, dynamic pricing, and information overload. BharatBuy is engineered to address these challenges by providing an intelligent e-commerce assistant that unifies multi-platform searches, performs value-based product scoring, and offers automated price tracking. By bridging the gap between raw data aggregation and actionable consumer insights, BharatBuy streamlines the purchasing process, empowering users to make informed, budget-conscious decisions in an increasingly complex digital economy.

To realize this vision, the platform incorporates a robust, full-stack architecture built on modular PHP and a normalized MySQL relational database, coupled with dynamic, responsive frontend interfaces. BharatBuy integrates live multi-vendor product crawling with native Indian Rupee (₹) normalization, interactive historical price trend charting, and an automated background alert engine that delivers real-time SMTP price-drop notifications directly to users. In addition, the system features a conversational AI shopping assistant powered by Google Gemini with multi-model failover resilience and native speech synthesis (Read Aloud), enabling users to receive personalized, context-aware product recommendations effortlessly. Together, these technological components transform BharatBuy from a conventional price comparison portal into an interactive, end-to-end smart shopping companion that delivers maximum market transparency, convenience, and financial savings to online shoppers.

1.2 Problem Definition

The development of **BharatBuy** addresses the following critical challenges faced by online consumers today:

1. E-commerce Platform Fragmentation: The online retail space is highly fragmented. To purchase a single product, consumers must manually visit multiple websites (such as Amazon, eBay, and Flipkart) to compare prices. This repetitive process leads to information overload, user fatigue, and a high amount of time wasted switching between platforms.
2. The "Cheapest Price" Trap: Most comparison tools only sort items by the lowest price. However, the cheapest option is often sold by a low-rated merchant, has poor customer reviews, high hidden shipping fees, or is out of stock. Consumers lack a

balanced scoring system that evaluates the overall value of the deal, leading to buyer's remorse.

3. Inability to Track Rapid Price Fluctuations: E-commerce prices fluctuate continuously based on demand, stock levels, and promotional events. It is impossible for a user to manually monitor these price shifts, resulting in missed savings on high-ticket items.
4. No Cross-Platform Competitor Price Drop Alerts: Individual online stores only notify users about their own internal deals and will never alert a user if a competitor drops their price. Consumers lack a platform-agnostic tracker that monitors competitor pricing on their behalf.
5. Scattered and Disorganized Wishlists: Shoppers must maintain separate, isolated wishlists on multiple websites. There is no centralized place to organize saved items across different platforms, make side-by-side comparison calls, and manage price alerts.
6. Lack of Instant, Interactive Shopping Assistance: Traditional e-commerce websites offer static FAQ pages that do not help users solve active doubts or navigate complex features. Consumers lack a conversational assistant to answer questions about recommendations and platform functionality in real-time.
7. Search Noise and Accessory Pollution: When consumers search for a specific high-value product (such as a smartphone like "Realme P3x Pro" or "OnePlus 10T"), the search result pages on major e-commerce platforms are heavily polluted with unrelated low-value accessories like phone cases, tempered glass screen guards, charging cables, and sleeves. This "noise" obscures the actual product the user is looking to buy, making price comparison highly frustrating.
8. Shared Device Login Friction in Households: In typical Indian households, multiple family members often share a single computer or tablet to browse and shop online. Constantly logging out, re-typing email addresses, and entering credentials creates significant usability friction and leads to shared accounts, which compromises the personalization of wishlists, search history, and price alert logs.

1.3 Existing System & Its Drawbacks

1. **Manual Multi-Site Comparison:** When a user wants to buy a product (e.g., a smartphone or a laptop), they open multiple tabs in their web browser for different e-commerce websites (such as Amazon, Flipkart, eBay, or Reliance Digital). The user manually types the product name in each search bar, scrolls through the results, notes down the prices, delivery charges, and seller ratings, and then makes a mental comparison to decide which platform offers the best option.
2. **Traditional Price Aggregators:** Standard price comparison platforms aggregate prices from multiple sources and present them in a list. However, their core logic is fundamentally unidimensional: they sort products strictly from the lowest price to the highest price.

Drawbacks and Bottlenecks of the Existing System : While these methods are widely used, they have significant operational bottlenecks, which are detailed below:

High Cognitive Load and Tab Fatigue : Manual comparison is time-consuming and exhausting. Users must constantly toggle between tabs, keep track of volatile prices, calculate taxes or delivery fees, and remember which seller had the best rating. This repetitive process leads to decision fatigue and often results in sub-optimal purchases.

The "Cheapest Price" Trap (Ignoring Quality Signals) : Traditional comparison sites only prioritize the cheapest absolute price. They fail to analyze critical metadata such as:

- Seller Reliability: A listing might be extremely cheap because it is posted by a new, unverified, or poorly-rated seller.
- Review Volume: A product with a 5-star rating based on a single review is ranked higher than a 4.7-star product with 10,000 reviews.
- Stock Status: Out-of-stock listings are often displayed as "cheap deals," leading to frustration when the user redirects to purchase.

Fragmented Wishlists and Data Silos: Currently, consumers must maintain separate, isolated wishlists on each individual shopping platform (e.g., one on Amazon, one on Flipkart). There is no centralized dashboard where a user can save a product and monitor its price fluctuations across all competing portals simultaneously.

Lack of Cross-Platform Price Tracking and Automation : Existing e-commerce portals only notify users about their own internal price drops. If a user sets an alert on Amazon for a laptop, they will never receive a notification if eBay or Flipkart drops the price of that same laptop below their target.

Unfiltered Search Noise and Accessory Pollution : When searching for high-value items (e.g., a specific smartphone model like "OnePlus 11R"), standard search queries return hundreds of low-value accessories such as phone cases, tempered glass, charger cables, and skin stickers. Users have to manually filter out this noise to find the actual device, making comparison frustrating.

Friction on Shared Household Devices : In typical Indian households, family members often share a single computer or tablet to browse and shop online. In the existing system, users must manually log out and re-type login credentials to switch profiles. This friction results in users sharing a single account, which mixes search histories, wishlists, and price alerts.

Complete Dependence on Upstream API Stability : Most existing comparison tools rely entirely on live external API calls or web scrapers. If a merchant website (e.g., Amazon or eBay) is temporarily rate-limiting requests or experiencing downtime, the comparison portal crashes or displays empty, broken pages to the user, lacking any self-healing fallback mechanisms.

1.4 Need for the New System

Project Background and Context

The rapid expansion of the digital economy has transformed the retail landscape globally, and particularly in India. With initiatives like "Digital India," the widespread adoption of high-speed mobile internet, and the revolution of unified payment interfaces (UPI), e-commerce has shifted from a luxury to an everyday necessity. Millions of Indian consumers now purchase electronics, apparel, household items, and groceries online.

However, this explosive growth has led to a highly fragmented market. The same product is frequently listed across multiple online shopping platforms—such as Amazon, eBay, Flipkart, and Walmart—at varying prices, shipping fees, delivery times, and seller terms. E-commerce portals employ dynamic pricing algorithms that adjust rates based on demand, search history, inventory levels, and competitor behavior. Consequently, consumers are faced with the daunting task of manually tracking and comparing prices to ensure they do not overpay, creating a significant cognitive load.

The Price Comparison Dilemma

Traditional price comparison websites only solve a fraction of this problem. They typically pull pricing data and sort listings from cheapest to most expensive. This approach is fundamentally flawed because it ignores crucial trust and quality signals. A product that appears cheap might have an extremely low customer rating, a small volume of reviews, high shipping costs, or be completely out of stock.

In e-commerce, a purchase decision is based on a combination of cost, quality, and reliability. Without an aggregated tool that evaluates these factors collectively, consumers often fall victim to low-quality products or fraudulent listings. There is a clear need for an intelligent assistant that not only aggregates products but also analyzes metadata to recommend the overall best deal.

The Branded Identity

BharatBuy is designed specifically to fill this gap. Styled as a premium, localized, and intelligent shopping assistant, the name *BharatBuy* represents a tool created to empower Indian consumers to navigate e-commerce platforms confidently and make smart financial decisions. By integrating real-time web scrapers, a custom evaluation algorithm, and state-of-the-art AI, the application acts as a personal shopping consultant working on behalf of the buyer. It provides transparency, saves time, and helps families optimize their monthly household budgets.

Technological Innovation

From a software engineering perspective, BharatBuy is a full-stack web application designed on a three-tier model using PHP and MySQL. It introduces several key technical innovations:

PHP and MySQL were selected for their ubiquity, scalability in academic environments, and native compatibility with the XAMPP development environment, providing a robust backend.

1. Dynamic Scraper Orchestrator: Merges multi-source asynchronous queries, executes normalization scripts, and handles fallback mock sets smoothly when upstream APIs are throttled or offline.
2. Value Score Recommendation Algorithm: Applies mathematical weights to different product parameters to output a single, easy-to-understand value rating out of 100, highlighting the "Best Overall Deal."
3. Automated Price Tracking Engine: Monitors user-specified target thresholds and integrates with SMTP mail servers using Google App Passwords to deliver instant, real-time email alerts.
4. Conversational LLM Integration: Embeds the Google Gemini API to parse natural language shopping queries, suggest products, and provide a conversational user experience.
5. Session Account Selector: Redesigns the entry portal with a Gmail-style account selector to make profile switching secure and frictionless.

1.5 Objectives of the Proposed System

The primary objectives of the BharatBuy project are:

- Unified Search Aggregation: To develop a centralized search interface that queries multiple active e-commerce platforms (like Amazon and eBay) simultaneously, eliminating the need for manual site-to-site searches. This mechanism saves the user from opening dozens of browser tabs by unifying raw API search results. The database acts as a central aggregator, normalizing various product data models into a clean, uniform list that is displayed instantly to the user.
- Dynamic Data Scraping & API Connections: To build a robust web scraping and API connector orchestrator that retrieves real-time pricing, ratings, reviews, and stock availability of products. By utilizing structured cURL requests, API providers, and scraper fallbacks, the system guarantees up-to-the-minute product information. If live external APIs fail or are throttled, the engine utilizes a high-fidelity MockProvider fallback to maintain system functionality and prevent empty result pages.

- Intelligent Value Recommendation Scoring: To implement a custom, weighted mathematical algorithm (Value Score out of 100) that ranks deals by overall value (balancing price, discount, rating, and review count) rather than just listing the cheapest option. This recommendation engine weighs product price at 40%, customer rating at 30%, and review count volume at 30%. It automatically tags the highest-scoring product as the "Best Overall Deal" at the top of the search grid, shielding users from buying cheap, low-rated items.
- Historical Price Tracking & Trend Charts: To design a historical database logger that records product price variations over time, displaying them to the user via interactive trend charts. The database registers every price fluctuation captured during searches. The details page then formats this data and feeds it to Chart.js on the frontend, drawing a smooth, interactive line graph. If the product is brand new and has no history yet, the system draws a clean horizontal baseline matching the current live price to keep the user interface looking complete.
- Real-time Price Drop Alerts: To enable users to set custom target price thresholds for any product, triggering automatic database trackers. Shoppers can input their desired purchase price on any product page. This registers a tracking hook in the product_trackers table. Whenever the system runs checks, it matches the target threshold against the current live price. If the price drops to or below the target, the trigger fires, log-notifies the user, and deactivates the tracker to prevent duplicate alerts.
- Automated SMTP Email Notifications: To integrate secure SMTP mail delivery (via Gmail App Passwords) that instantly emails users when a tracked product's price drops to or below their target threshold. Powered by PHPMailer, the email system connects securely via TLS encryption to Gmail's servers using credentials stored safely in a .env file. It sets the character set encoding to UTF-8, ensuring that emojis (like 🔥) and special symbols in the subject line render cleanly on all mobile and desktop email clients.
- Conversational AI Chatbot: To integrate a natural language processing assistant powered by the Google Gemini API to answer shopping queries, guide platform usage, and recommend search prompts. The chatbot is embedded globally across all pages and uses custom system prompt engineering. If the external Gemini API is unreachable or rate-limited, the system automatically falls back to local database keyword matching, ensuring the chat interface remains functional and responsive at all times.
- Secure User Session & Account Management: To implement safe account creation, hashed passwords, session handling, and a personalized profile dashboard containing wishlist items and alert history. The system hashes user passwords using the secure bcrypt algorithm. It tracks active user sessions to protect dashboard access, and integrates a customized, Gmail-style account selector card on the login screen to allow quick, safe profile switching for saved local accounts.
- Premium Responsive User Interface: To design a modern, visually stunning frontend using vanilla CSS that adapts seamlessly across mobile screens, tablets, and desktops. The UI utilizes custom CSS variables, elegant transitions, micro-animations (like a shaking notification bell), glassmorphism, and unified button styles. All grids use auto-fill alignments to prevent product cards from stretching awkwardly, ensuring a premium e-commerce look.

- Academic & Scalable Architecture: To structure a clean three-tier PHP/MySQL architecture with modular code files, safe inputs, and robust database query handlers ready for academic evaluation. By separating database connections (db.php), configuration constraints (config.php), visual headers/footers, and AJAX endpoints, the project follows strict software engineering practices. All inputs are sanitized using PDO prepared statements to block SQL injection, meeting all criteria for a BCA Semester III User Defined Project.

1.6 Scope & Core Modules

This section outlines the functional boundaries of the BharatBuy application (what the system is designed to do and what is excluded) and details the core software components that make up its architecture.

The scope of BharatBuy is centered on providing a centralized, intelligent, and secure comparison environment for online shoppers.

In-Scope Features

Centralized E-commerce Aggregation: Programmatic search querying across multiple platforms (e.g., Amazon, eBay, and a Mock fallback) from a single user dashboard.

Intelligent Product Comparison & Scoring: Automated calculation of a multidimensional "Value Score" for each search result to rank products.

Price Tracking & Dynamic Alerting: Archiving price histories, rendering interactive trend charts, and sending automated SMTP email alerts when target prices are met.

Conversational Navigation: A global AI-powered chatbot assistant to answer user questions, explain recommendations, and suggest keywords.

Session Management: Hashed password registration, secure login, and a frictionless Gmail-style account selector for shared household profiles.

Search Noise Control: An active strict keyword filtering system to isolate main products from low-value accessories.

Out-of-Scope Features

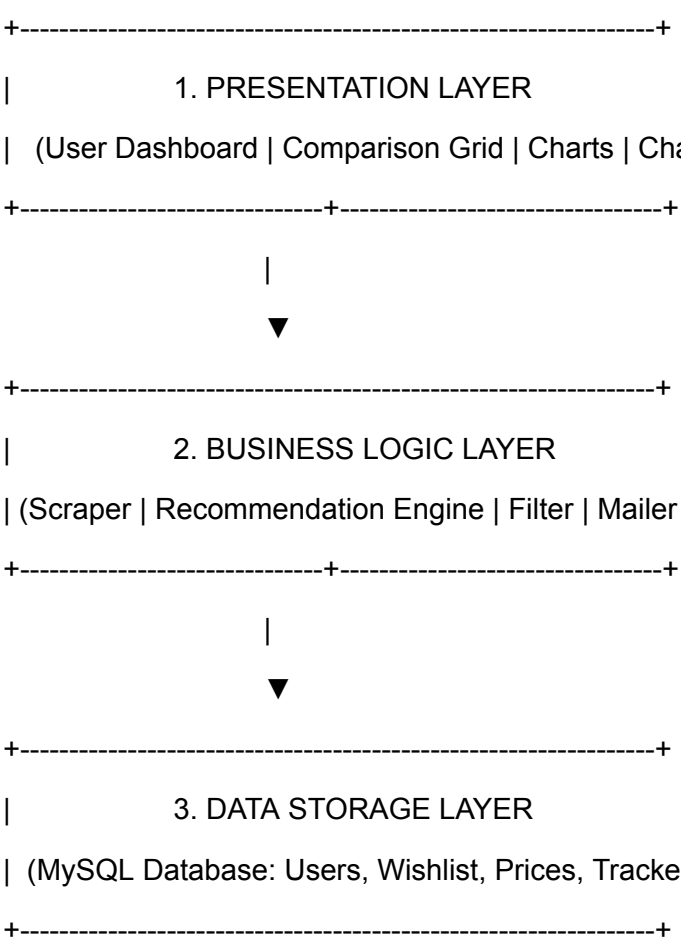
Payment Processing: The system does not handle financial transactions, credit cards, or digital wallets.

E-commerce Fulfillment & Shipping: BharatBuy does not sell or ship products. Instead, it acts as a referral portal, redirecting the user via outbound links to complete the purchase on the respective vendor’s website.

Inventory Control: The system does not manage merchant product stock; it only displays the real-time stock status retrieved from the external source.

Core System Components

The architecture is divided into three primary components, following the standard three-tier model:



Presentation Layer (Frontend UI)

This component provides the interface for user interaction, designed using responsive HTML5 and Vanilla CSS3 with custom modern styling (glassmorphism, subtle micro-animations):

Interactive Dashboard: Displays search controls, recent searches, wishlist items, and notifications in one place.

Comparison Grid: Formats product listings dynamically with a "Best Overall Deal" highlight card at the top.

Detail Visualization: Features an interactive price-trend line chart powered by Chart.js on the product detail pages.

Global Chatbot Widget: An overlay window situated at the bottom-right corner of all pages to handle user conversations.

Business Logic Layer (Backend PHP Engine)

This is the core functional engine of the application, handles data processing, API interactions, and system behaviors:

API Connector & Scraper Orchestrator: Manages external cURL connections to third-party endpoints. It standardizes raw data models and automatically runs the self-healing MockProvider if APIs fail.

Weighted Recommendation Engine: Calculates the Value Score (0 to 100) using the weighted formula ($40%\text{ Price} + 30%\text{ Rating} + 30%\text{ Review Volume}$).

Noise Filter Module: Scans titles of incoming search results and filters out accessory keywords when Strict Mode is active.

Mailer & Alert Trigger Engine: Monitors active user trackers, checks if the live price is at or below the target threshold, and handles secure SMTP mailing via PHPMailer.

AI Chatbot Controller: Sends natural-language prompts to the Google Gemini API and handles the database fallback search when the API is rate-limited.

Data Storage Layer (MySQL Relational Database)

This component manages persistent storage across 10 relational tables:

Shopper Data: Stores user credentials, session settings, and account profiles.

Shopping Logs: Archives wishlist entries, recent searches, and price alert thresholds.

Analytical Records: Tracks product price history logs to feed the charting engine.

System Diagnostics: Stores metrics for API response times and connection errors (provider_logs and chatbot_logs) to assist developers in monitoring application health.

2. Project Profile & Feasibility

2.1 Project Profile (Team, Tools, Target Audience)

The project profile provides a concise, high-level summary of the BharatBuy application, outlining its key attributes, technical stack, development environment, and course context.

Project Metadata Table

Project Attribute	**Project Details**
Project Title	BharatBuy
Project Type	User Defined Project (UDP)
Academic Course	Bachelor of Computer Applications (BCA)
Academic Semester	Semester III
Platform / Target Environment	Web-based (Responsive for Desktop, Tablet, and Mobile)

Primary Language & Backend	PHP 8.x
Frontend Technologies	HTML5, CSS3, JavaScript (Vanilla ES6), AJAX (Fetch API)
Database Management System	MySQL
Local Host Server	XAMPP (Apache Web Server, MySQL Database)
Development IDE / Editor	Antigravity IDE
Database Management UI	phpMyAdmin
Key External Libraries	Chart.js (Data Visualization), PHPMailer (Secure SMTP Mail)
AI / LLM Integration	Google Gemini API (Conversational Helper) AND eBay API
Version Control System	Git & GitHub
Target Audience	BCA Students and Evaluators

System Modules Overview

To help the evaluator understand how the software is structured, the project is divided into the following functional modules:

User Authentication & Session Module: Handles secure registration (using bcrypt hashing), login authentication, dashboard access guards, and the session account chooser.

Search Aggregation & Scraper Module: Integrates external e-commerce data queries, runs data model normalization, and manages the self-healing MockProvider fallback.

Recommendation Engine Module: Processes price, rating, and review volumes to generate a weighted Value Score and flags the "Best Overall Deal".

Wishlist & History Module: Handles user bookmarks (with auto-healing image paths) and tracks search keywords.

Analytical Charting Module: Formats price variation logs over time and feeds them to Chart.js to render price drop timelines.

Background Alerting & Mailer Module: Monitors target thresholds, logs alerts, and fires automated SMTP emails via PHPMailer.

Conversational Assistant Module: Embeds the conversational interface, routes queries to the Gemini API, and falls back to a database FAQ match when offline.

2.2 Hardware and Software Requirements

The system requires a modern computer capable of running local web server environments (such as XAMPP with Apache and MySQL) alongside standard browser software for interface evaluation and backend development tools like Antigravity IDE and phpMyAdmin.

2.3 Feasibility Study (Technical, Operational, Economic)

Technical feasibility is demonstrated through the lightweight PHP and MySQL architecture combined with cURL scraping and Google Gemini API integration. Operational feasibility is ensured via intuitive dashboard layouts and clear automated alerts, while economic feasibility is maintained using open-source web technologies and standard free-tier APIs.

2.4 Required Determinants

2.4.1 Functional Requirements (FR)

1. **FR-01: Multi-Platform Product Search Aggregation -** The system shall query multiple third-party e-commerce APIs/scrapers (e.g., Amazon, eBay) concurrently to aggregate live product listings based on user-entered keywords.
2. **FR-02: Data Normalization & Currency Standardization -** The backend shall normalize raw external product data schemas and convert multi-currency pricing into Indian Rupees (₹) for uniform presentation.
3. **FR-03: Weighted Value Score Calculation -** The system shall calculate a Value Score (0–100) using a weighted algorithm (40% Price, 30% Customer Rating, 30% Review Volume) to identify and highlight the "Best Overall Deal".
4. **FR-04: Strict Keyword Accessory Filtering -** The application shall filter out low-value accessories (cases, screen guards, cables) when Strict Search Mode is enabled for target high-value search items.
5. **FR-05: Price History Logging & Interactive Visualization -** The database shall log historical price variations per product and render interactive trend lines using Chart.js on the product detail page.
6. **FR-06: Custom Target Price Alert Hooks -** Users shall be able to set target price thresholds for any tracked item, registering automated hooks in the tracking database table.
7. **FR-07: Automated SMTP Email Notifications -** The system shall send automated price-drop email notifications via PHPMailer (using secure TLS/Google App Passwords) when live prices reach or drop below target thresholds.
8. **FR-08: Conversational AI Shopping Assistant -** The application shall integrate an AI assistant powered by Google Gemini to parse user prompts, suggest search terms, and provide product explanations.
9. **FR-09: Multi-Model API Failover & Local FAQ Matching -** If external APIs or LLMs fail or hit rate limits, the system shall seamlessly transition to a local MockProvider and database keyword-matching fallback engine.
10. **FR-10: Secure Session & Household Account Chooser -** The system shall store user passwords using bcrypt hashing, manage secure sessions, and offer a Gmail-style account selector for quick profile switching on shared household devices.

2.4.2 Non-Functional Requirements (NFR)

- **Performance:** Aggregated search responses shall complete within acceptable response thresholds using asynchronous queries, with local AJAX endpoints responding in under 500ms.

- **Security:** All user credentials must be stored with strong password hashing (bcrypt), sensitive API tokens must be isolated in environment files (.env), and database queries must use prepared statements (PDO) to protect against SQL injection.
- **Reliability:** The backend must demonstrate high operational resilience by maintaining application usability during third-party service downtime or API rate-limiting via self-healing fallback handlers.
- **Accessibility:** The interface shall feature a modern, responsive design using dynamic UI controls, clear typography, and integrated text-to-speech voice synthesis (Read Aloud) for improved accessibility across devices.
- **Maintainability:** The codebase shall adhere to a modular 3-tier software architecture with strict separation between user interfaces, server-side processing scripts, and database operations to support future system extensions.

2.5 Targeted Users

The BharatBuy platform is designed to serve a diverse group of users, ranging from individual online shoppers seeking maximum savings to system administrators maintaining platform health and operations. By analyzing user workflows, cognitive loads, and decision-making habits, the platform delivers tailored solutions across three primary user segments.

Everyday Bargain Hunters & Deal Seekers (Primary Segment)

Profile Definition

- Students, working professionals, homemakers, and budget-conscious consumers located in metro areas and Tier 1–3 Indian cities.
- Shoppers focused on key categories including mobile devices, consumer electronics, fashion, audio gear, and household goods.
- Frequent buyers on dominant e-commerce portals such as Amazon India, Flipkart, Reliance Digital, and Croma.

Operational Bottlenecks & Consumer Pain Points

- **High Cognitive Load and Tab Fatigue:** Users open up to 15 browser tabs to manually review pricing across various platforms, causing severe decision fatigue and wasted time.
- **Volatile Sale Pricing:** Dynamic rate changes during seasonal promotion events (e.g., Big Billion Days or Great Indian Festival) obscure whether a advertised drop represents genuine savings.
- **Missed Price Drops:** Manual monitoring is impractical, leading consumers to miss short-lived discount drops and special promo codes.

System Solutions Provided by BharatBuy

- **Unified Search Aggregation:** A centralized dashboard allows users to submit a single query to retrieve real-time side-by-side listings across competing platforms.
- **Automated Price Drop Alert Engine:** Users can specify target thresholds (e.g., "Alert me when this item drops below ₹24,000"), triggering background tracking hooks and automatic SMTP email alerts upon price hits.
- **Centralized Wishlist Management:** Eliminates data silos by allowing users to organize saved products across multi-platform vendors within a single unified profile.

Tech Enthusiasts & Hardware Analysts (Secondary Segment)

Profile Definition

- Technology hobbyists, software engineers, reviewers, and buyers purchasing premium hardware (laptops, graphics cards, flagship smartphones, and consoles).
- Users focused on seller reliability, valid brand warranties, international vs. domestic variants, and granular specification details.

Operational Bottlenecks & Consumer Pain Points

- **The "Cheapest Price" Trap & Unverified Sellers:** Listings displaying low prices often hide low merchant ratings, absent warranty terms, or open-box items passed off as new.
- **Analysis Paralysis:** Evaluating competing high-tier flagship devices (e.g., iPhone 16 Pro vs. Samsung S24 Ultra) without consolidated metrics is difficult.
- **Cross-Border FX Complexity:** Converting foreign currency listings (USD/EUR to INR) while factoring seller trustworthiness introduces calculation friction.

System Solutions Provided by BharatBuy

- **Intelligent Value Score Algorithm:** Computes a weighted metric (40% Price, 30% Rating, 30% Review Volume) to highlight the best overall deal instead of basic price sorting.
- **Conversational AI Shopping Assistant:** Employs Google Gemini API integration to parse technical prompts, compare specifications, and provide contextual recommendations.
- **Dynamic Currency Normalization:** Automatically standardizes international listings into Indian Rupees (₹) to support accurate price evaluation.

System Administrators & Platform Operators (Internal Operations)

Profile Definition

- System administrators, support teams, and technical engineers maintaining platform health, uptime, and data accuracy.

Operational Bottlenecks & System Challenges

- **Third-Party API Instability & Rate Limits:** Upstream merchant scraper endpoints experience frequent rate-limiting, schema changes, or sudden downtime.
- **Unmonitored Latency:** API processing delays and LLM response times require constant monitoring to prevent performance degradation.
- **Account & Notification Oversight:** Ensuring efficient user profile management, monitoring automated tracking hooks, and maintaining SMTP delivery health.

System Solutions Provided by BharatBuy

BharatBuy equips platform administrators with an internal diagnostic suite to monitor and manage system performance:

- **Provider Telemetry Diagnostics (provider_logs):** Tracks HTTP response codes, latency metrics, and failover rates across scraper integration endpoints.
- **Conversational AI Analytics (chatbot_logs):** Audits Gemini API response times, fallback occurrences, and exception details.

- **User & Tracker Management (users, product_trackers):** Enables administrators to adjust user permissions, manage profiles, and monitor active automated price tracking hooks across the database.

Target User Group Comparison Matrix

The following table provides a comprehensive summary of the target user groups, their primary objectives, critical pain points, and the core BharatBuy features engineered to serve them:

User Group	Primary Goal	Key Pain Point	Core BharatBuy Feature
Bargain Hunters	Find lowest prices and genuine discount drops across platforms.	Tab fatigue, manual tracking, missed price drops.	Unified Search, Target Price Hooks, SMTP Email Alerts.
Tech Enthusiasts	Evaluate product value, seller credibility, and technical specs.	Cheap low-quality deals, accessory noise, currency friction.	Value Score Algorithm, Strict Search Filter, Gemini AI Assistant.
System Administrators	Maintain system uptime, monitor API health, and manage accounts.	Upstream API instability, rate limits, unmonitored latency.	Telemetry Diagnostics (provider_logs, chatbot_logs), Mock Fallback.

3. Assumptions and Constraints

This section outlines the baseline assumptions made during the design of BharatBuy and the technical/operational constraints that restrict its implementation.

3.1 Assumptions

Constant Internet Connectivity: The application assumes that both the hosting server (XAMPP) and the user's device have a continuous internet connection. This is necessary to fetch product details from external e-commerce APIs, communicate with the Google Gemini API, and send email alerts.

Modern Web Browser Support: It is assumed that the end-users access the system using modern web browsers (e.g., Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge) that support HTML5, CSS3 variables, ES6 JavaScript, and the AJAX Fetch API.

Accuracy of Third-Party Data: The system assumes that the product data (prices, ratings, review counts, stock status) retrieved from e-commerce APIs is accurate and matches what is displayed on the parent websites.

SMTP Server Permissions: It is assumed that the configured SMTP server credentials (such as Google App Passwords) remain valid and authorized to send outgoing alert emails.

Data Privacy and Ethics: The system prioritizes security and does not store sensitive personal financial information, such as credit card numbers or banking credentials.

3.2 Constraints & Dependencies

API Rate Limits: Third-party APIs (SerpApi, eBay Browse API, and Google Gemini API) have strict rate limits on their free tiers. This constrains the volume of searches and chatbot queries that can be executed in a given hour or day.

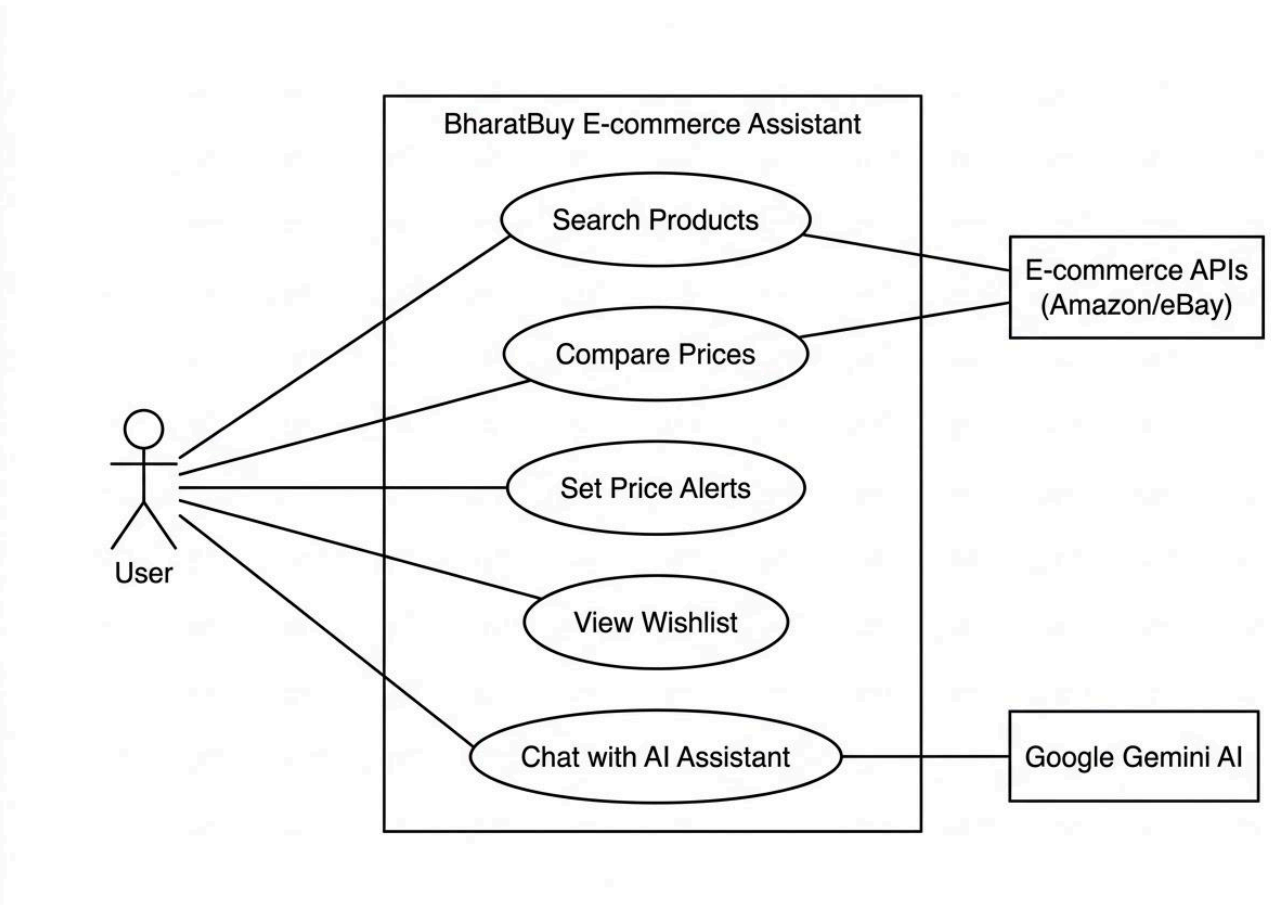
API Structure Volatility: The application is highly dependent on the response format of external APIs. If an external vendor modifies their JSON payload structure, the system's parsing and normalization logic will break until manual code updates are made.

Local Server Execution Bounds: Since the system runs in a local environment (XAMPP on localhost), automated price tracking and background alert processes rely on the local Apache server being actively turned on. It cannot run as a continuous cloud service unless hosted online.

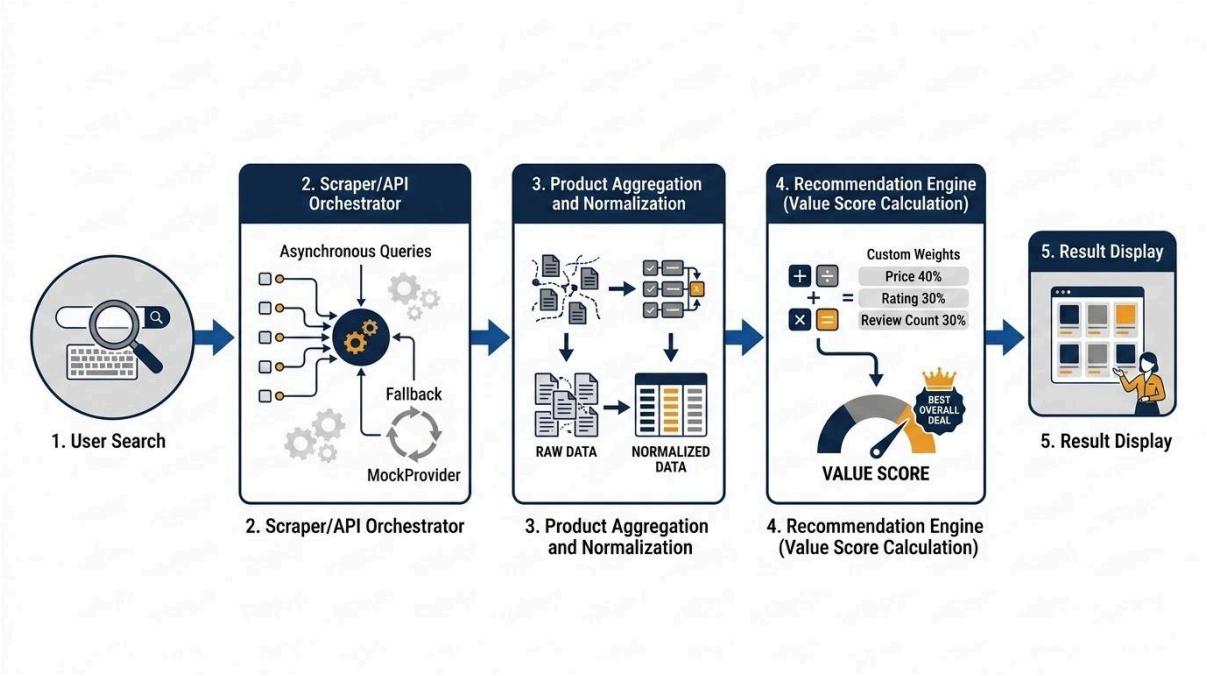
Transaction Boundary: The system is strictly constrained from processing payments or managing shopping carts. It must redirect the user to the external merchant's website to complete the actual checkout.

4. System Analysis & Design (UML Modeling)

4.1 Use Case Diagram & Descriptions



4.2 System Flowchart / Activity Diagram



5. Database Design

5.1 Entity-Relationship (ER) Diagram

[Placeholder for ER Diagram]

5.2 Data Dictionary (smart_shopping_assistant Database)

1. Users Table

Stores registered shopper credentials, personal settings, and system roles.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique identifier for each user
full_name	VARCHAR(100)	-	NULL	Full name of the registered user
email	VARCHAR(150)	UNIQUE	None	Unique email address used for login and notifications
password	VARCHAR(255)	-	None	Securely hashed password using bcrypt algorithm
alerts_enabled	TINYINT(1)	-	1	Toggle setting for email notifications (1 = Enabled, 0 = Disabled)

created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Date and time when user account was created

---

2. product_prices Table

Stores aggregated product listings, current price metrics, and seller metadata.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique product deal record identifier
product_name	VARCHAR(255)	-	None	Full product title retrieved from supplier
website	VARCHAR(100)	-	None	Merchant store platform (e.g., Amazon, eBay, Flipkart)
current_price	DECIMAL(10,2)	-	0.00	Live selling price normalized in Indian Rupees (₹)
original_price	DECIMAL(10,2)	-	NULL	MSRP / original price before discount
discount_rate	INT	-	0	Calculated discount percentage (0 to 100)
rating	DECIMAL(3,2)	-	0.00	Average customer rating score (0.00 to 5.00)
review_count	INT	-	0	Total volume of user product reviews
image_url	TEXT	-	NULL	Source URL for product image thumbnail
product_url	TEXT	-	None	Outbound referral link to merchant product page
is_out_of_stock	TINYINT(1)	-	0	Availability status (1 = Out of Stock, 0 = In Stock)
category	VARCHAR(100)	-	NULL	Product classification category name
created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when product listing was fetched
updated_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Auto-updated timestamp on modification

---

3. product_trackers Table

Manages active price drop hooks and user target threshold triggers.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique tracker ID
user_id	INT	FOREIGN KEY	None	Links to users.id (Cascades on delete)
product_id	INT	FOREIGN KEY	None	Links to product_prices.id (Cascades on delete)
target_price	DECIMAL(10,2)	-	None	Desired price threshold defined by user
is_active	TINYINT(1)	-	1	Tracker state (1 = Active, 0 = Alert Triggered & Deactivated)
created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when tracker was registered

---

4. price_history Table

Logs historical price variations over time to feed Chart.js visual trend lines.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique history log entry ID
product_id	INT	FOREIGN KEY	None	Links to product_prices.id (Cascades on delete)
price	DECIMAL(10,2)	-	None	Price value recorded at time of query
recorded_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when price check was captured

---

5. wishlist Table

Stores bookmarked product deals per user profile.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique wishlist entry ID
user_id	INT	FOREIGN KEY	None	Links to users.id (Cascades on delete)
product_name	VARCHAR(255)	-	None	Title of the saved product

website	VARCHAR(100)	-	NULL	Source merchant platform name
current_price	DECIMAL(10,2)	-	0.00	Saved current price in INR
product_url	TEXT	-	None	Outbound referral URL
image_url	TEXT	-	NULL	Image thumbnail URL (auto-repaired on load if broken)
created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when item was bookmarked

---

6. search_history Table

Logs recent user search queries for quick dashboard access.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique search log ID
user_id	INT	FOREIGN KEY	None	Links to users.id (Cascades on delete)
product_name	VARCHAR(255)	-	None	Raw search string query entered by user
searched_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp of search query execution

---

7. user_notifications Table

Stores in-app bell notification messages triggered by system events.

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique notification log entry ID
user_id	INT	FOREIGN KEY	None	Links to users.id (Cascades on delete)
title	VARCHAR(150)	-	None	Notification header subject title
message	TEXT	-	None	Message body content detailing price drop
product_id	INT	FOREIGN KEY	NULL	Links to product_prices.id (Set NULL on delete)
created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when notification was issued

---

8. chatbot & chatbot_logs Tables

The chatbot table stores keyword FAQ responses for local fallbacks. The chatbot_logs table records API latency and runtime execution errors.

chatbot Table Schema:

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique FAQ record ID
keyword	VARCHAR(150)	-	None	Matching keyphrase (e.g., "price alert")
answer	TEXT	-	None	Predefined system answer text

chatbot_logs Table Schema:

Field	Type	Key	Default	Description
id	INT	PRIMARY KEY	AUTO_INC	Unique interaction log ID
user_id	INT	FOREIGN KEY	NULL	Links to users.id (Set NULL on delete)
source	VARCHAR(20)	-	None	Response source ('gemini', 'fallback', 'rate_limit')
response_time_ms	INT	-	0	Response latency measured in milliseconds
error_message	TEXT	-	NULL	Captured API exception details if call failed
created_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp of chatbot query execution

---

9. api_cache & provider_logs Tables

The api_cache table stores temporary search payload JSON responses to prevent rate-limiting. The provider_logs table monitors third-party API connection statuses.

api_cache Table Schema:

Field	Type	Key	Default	Description
cache_key	VARCHAR(64)	PRIMARY KEY	None	MD5 hash key generated from search query
response_json	LONGTEXT	-	None	Cached API JSON response payload

expires_at	DATETIME	-	None	Expiration timestamp for cache invalidation

provider_logs Table Schema:

Field	**Type**	**Key**	**Default**	**Description**
id	INT	PRIMARY KEY	AUTO_INC	Unique provider log entry ID
provider_name	VARCHAR(100)	-	None	Class name of provider (e.g., AmazonNinjaProvider)
status	INT	-	200	HTTP response status code (200 = Success, 500 = Error)
response_time_ms	INT	-	0	Scraper query round-trip time in milliseconds
error_message	TEXT	-	NULL	Error exception message if endpoint query failed
logged_at	TIMESTAMP	-	CURRENT_TIMESTAMP	Timestamp when API provider status was logged

users Table

Stores registered shopper credentials and settings.

Column Name	Data Type	Size	Description	Example
id	INT	11	Unique identifier for each user (Primary Key)	1
full_name	VARCHAR	100	Full name of the user	Param Shah
email	VARCHAR	150	Registered email address (Unique)	param@example.com
password	VARCHAR	255	Securely hashed password	$2y$10$e8...
alerts_enabled	TINYINT	1	Toggle for email notifications (1 = Enabled)	1
created_at	TIMESTAMP	-	Date and time of user registration	2026-08-17 10:00:00

---

product_prices Table

Stores current product deals aggregated from e-commerce providers.

Column Name	Data Type	Key Type	Description

id	INT	PRIMARY KEY (Auto-Inc)	Unique identifier for each product deal
product_name	VARCHAR(255)	-	Full name / Title of the product
website	VARCHAR(100)	-	Source website (Amazon, eBay, Walmart, etc.)
current_price	DECIMAL(10,2)	-	Current listed price (in INR)
original_price	DECIMAL(10,2)	-	Original price before discount (in INR)
discount_rate	INT	-	Discount percentage (0-100)
rating	DECIMAL(3,2)	-	Average product customer rating (0.00 to 5.00)
review_count	INT	-	Total number of customer reviews
image_url	**TEXT**	-	Link to the product image
product_url	**TEXT**	-	Direct referral link to purchase on the merchant site
description	TEXT	-	Product description or key bullet points
is_out_of_stock	TINYINT(1)	-	Stock status (1 = Out of stock, 0 = In stock)
category	VARCHAR(100)	-	Product classification category (Laptops, Audio, Clothing, etc.)
created_at	TIMESTAMP	-	Timestamp when the product record was created
updated_at	TIMESTAMP	-	Auto-updated timestamp on modification

---

search_history Table

Logs user search terms to show recent searches.

Column Name	Data Type	Key Type	Description
id	INT	PRIMARY KEY (Auto-Inc)	Unique search log ID
user_id	INT	FOREIGN KEY	Links to users.id (Cascades on delete)

product_name	VARCHAR(255)	-	Raw search query string typed by the user
searched_at	TIMESTAMP	-	Timestamp of search execution

---

wishlist Table

Stores user-bookmarked items.

Column Name	Data Type	Key Type	Description
id	INT	PRIMARY KEY (Auto-Inc)	Unique wishlist item ID
user_id	INT	FOREIGN KEY	Links to users.id (Cascades on delete)
product_name	VARCHAR(255)	-	Name of the saved product
website	VARCHAR(100)	-	Source website of the product
current_price	DECIMAL(10,2)	-	Saved current price
product_url	TEXT	-	Purchase referral link
image_url	TEXT	-	Product image link (auto-repaired on load if broken)
created_at	TIMESTAMP	-	Timestamp when item was bookmarked

---

price_history Table

Logs historical price variations to draw Chart.js line charts.

Column Name	Data Type	Key Type	Description
id	INT	PRIMARY KEY (Auto-Inc)	Unique log entry ID
product_id	INT	**FOREIGN KEY**	Links to product_prices.id (Cascades on delete)
price	DECIMAL(10,2)	-	Price value logged at that point in time
recorded_at	TIMESTAMP	-	Timestamp when the price check was recorded

---

product_trackers Table

Manages active price drop trackers set by users.

Column Name	Data Type	Key Type	Description
id	INT	**PRIMARY KEY (Auto-Inc)**	Unique tracker ID
user_id	INT	**FOREIGN KEY**	Links to users.id (Cascades on delete)
product_id	INT	**FOREIGN KEY**	Links to product_prices.id (Cascades on delete)
target_price	DECIMAL(10,2)	-	Target threshold price input by the user
is_active	TINYINT(1)	-	Active toggle (1 = Tracking, 0 = Triggered and deactivated)
created_at	TIMESTAMP	-	Timestamp when the price alert was set

---

user_notifications Table

Stores notifications displayed in the header dropdown list.

Column Name	Data Type	Key Type	Description
id	INT	**PRIMARY KEY (Auto-Inc)**	Unique notification entry ID
user_id	INT	**FOREIGN KEY**	Links to users.id (Cascades on delete)
title	VARCHAR(150)	-	Header title (e.g. "🔥 Price Drop Alert!")
message	TEXT	-	Short message describing the price drop
product_id	INT	**FOREIGN KEY**	Links to product_prices.id (Nulls on delete)
created_at	TIMESTAMP	-	Timestamp when the notification was fired

---

chatbot Table

Stores predefined keyword-based FAQ responses for fallback chatbot matches. For instance, if a user queries with the keyword 'price alert', the system returns the answer: 'You can set price alerts by clicking the bell icon on any product page.'

Column Name	Data Type	Key Type	Description
id	INT	**PRIMARY KEY (Auto-Inc)**	Unique FAQ entry ID
keyword	VARCHAR(150)	**-**	Keyword trigger phrase (e.g. "how it works")
answer	TEXT	**-**	Pre-configured text answer matched to keyword

---

chatbot_logs Table

Tracks Gemini API request metrics, response times, and connection errors.

Column Name	Data Type	Key Type	Description
id	INT	**PRIMARY KEY (Auto-Inc)**	Unique chatbot log entry ID
user_id	INT	**FOREIGN KEY**	Links to users.id (Nulls on delete)
source	VARCHAR(20)	**-**	API classification source ('gemini', 'fallback', 'rate_limit')
response_time_ms	INT	**-**	Round-trip response time of request in milliseconds
error_message	TEXT	**-**	Exception details if the API call failed
created_at	TIMESTAMP	**-**	Timestamp when the chatbot interaction occurred

---

provider_logs Table

Tracks API statuses of e-commerce providers to track network success rates.

Column Name	Data Type	Key Type	Description
id	INT	**PRIMARY KEY (Auto-Inc)**	Unique log ID
provider_name	VARCHAR(100)	**-**	Class name of the provider (e.g. AmazonNinjaProvider)
status	INT	**-**	HTTP Response code (200 = Success, 500 = Error)
response_time_ms	INT	**-**	Query execution response time
error_message	TEXT	**-**	Details of exception if request failed
logged_at	TIMESTAMP	**-**	Timestamp of logging

---

6. System Evaluation

This section presents the advantages that the BharatBuy system offers to online shoppers, along with the technical and operational limitations inherent to its design.

6.1 Key Advantages of BharatBuy

Intelligent Value-Based Recommendations: Rather than sorting strictly by price, the system calculates a weighted Value Score based on price, customer ratings, and review volumes. This guides users to the best overall deals, preventing them from buying cheap but low-quality products.

Aggregated Time Efficiency: By consolidating search listings from multiple major shopping portals into a single dashboard, the system saves users time and eliminates browser tab clutter.

Clutter-Free Search (Strict Mode): The active keyword strictness filter isolates actual target products (like smartphones or laptops) from low-value search noise (cases, screen protectors, cables), providing a clean search layout.

Automated Cross-Platform Price Alerts: Users do not have to manually monitor price drops. The system tracks products in the background and sends secure SMTP email alerts via PHPMailer when a product price reaches the user's target threshold.

Frictionless Multi-User Access: The Gmail-style Account Chooser allows multiple family members sharing the same household device to switch accounts with a single click, keeping wishlists, search histories, and alert logs private.

Self-Healing Backend Architecture: If external APIs are rate-limited or go offline, the backend automatically transitions to a high-fidelity MockProvider fallback, keeping the application stable and responsive.

Interactive Historical Data: The integration of Chart.js renders clear, interactive price trend lines, allowing shoppers to visually check if a current discount is genuinely a good deal over time.

Conversational AI Guidance: The integrated Google Gemini AI chatbot provides natural-language assistance, helps users find products, and explains why a specific item is marked as the "Best Overall Deal".

6.2 Limitations of the System

API Rate Limiting: The system relies on third-party API free tiers (e.g., SerpApi, Google Gemini API). This limits the number of sequential searches and conversational prompts allowed within a specific timeframe.

Sensitivity to Schema Changes: If an external e-commerce provider changes its API output format or web structure, the system’s parsing and normalization logic will require manual updates to function again.

Synchronization Lag: The system updates pricing data when searches are triggered. Consequently, there may be a minor time delay between a price change occurring on the merchant's site and it reflecting in the BharatBuy database.

No Direct Purchase Capability: The system is not a direct marketplace. It cannot process payments or manage orders, and users must be redirected to external e-commerce websites to purchase items.

Local Hosting Dependency: Running on a local XAMPP environment means background price tracking only operates when the local computer and Apache web server are active.

7. Future Scope & Proposed Enhancements

- **Mobile Application & Push Notifications:** Develop native Android and iOS applications (using Flutter / React Native) to provide mobile-first price tracking, camera-based barcode/QR scanning in physical retail stores, and real-time device push notifications for instant flash-sale and price drop alerts.
- **Predictive AI & Time-Series Price Forecasting:** Implement machine learning time-series models (e.g., ARIMA, LSTM, or Prophet) on historical pricing datasets to forecast upcoming price trends, predict optimal buying times, and estimate drop probabilities during festive sale events (such as Diwali Sale, Great Indian Festival, and Big Billion Days).
- **Autonomous Coupon & Cashback Aggregator:** Build a lightweight Chromium-based browser extension and checkout helper that automatically discovers, verifies, and applies the highest-saving promo codes, bank card discounts, and UPI cashback offers at the final payment gateway.
- **Multi-Platform Vendor Expansion & WhatsApp/Telegram Bot Alerts:** Expand API crawling to regional Indian retail marketplaces (Flipkart, Myntra, Croma, Reliance

Digital, and Meesho) and integrate automated messaging webhooks via Telegram and WhatsApp Business APIs for instant deal notifications without opening the website.
- **Advanced Market Volatility & Seller Credibility Analytics:** Introduce interactive seller trust index scores and category-wide price volatility heatmaps to identify artificial price hikes before sale discounts ("fake discounts").

8. Conclusion

BharatBuy effectively solves one of the most critical consumer challenges in modern e-commerce—price fragmentation and volatile online discounts—by providing a centralized, intelligent, and transparent smart shopping ecosystem. By integrating live multi-vendor price aggregation, a weighted multi-factor value scoring engine, historical price trend analytics, and an interactive Gemini-powered conversational AI assistant with native voice synthesis (Read Aloud), the platform empowers consumers to make data-backed purchasing decisions in Indian Rupees (₹).

Throughout the project lifecycle, robust software engineering principles were implemented, including modular object-oriented PHP architecture, 3NF normalized MySQL relational database schemas, secure authentication and session control, multi-model API failover resilience, and asynchronous AJAX-driven user interfaces.

BharatBuy bridges the gap between static comparison websites and next-generation AI shopping assistants, establishing a resilient technical foundation for future expansions into mobile applications, predictive pricing algorithms, and automated browser-assisted shopping.

9. Bibliography & References

[1] Luke Welling and Laura Thomson, "PHP and MySQL Web Development", 5th Edition, Addison-Wesley Professional, 2016.
[2] Robin Nixon, "Learning PHP, MySQL & JavaScript: A Step-by-Step Guide to Creating Dynamic Websites", 6th Edition, O'Reilly Media, 2021.
[3] Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach", 4th Edition, Pearson, 2020.
[4] Jon Duckett, "JavaScript and JQuery: Interactive Front-End Web Development", Wiley Publishing, 2014.
[5] Google AI for Developers, "Gemini API Documentation & Quickstart Guide", https://ai.google.dev/gemini-api/docs, Accessed: August 2026.
[6] MDN Web Docs, "Web Speech API (SpeechSynthesis & SpeechRecognition)", Mozilla Developer Network, https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API, Accessed: August 2026.
[7] PHPMailer Community, "PHPMailer - Email Creation and Transfer Class for PHP", https://github.com/PHPMailer/PHPMailer, Accessed: August 2026.
[8] OpenWeb Ninja, "Realtime Amazon & E-Commerce Data Search API Documentation", https://openwebninja.com/, Accessed: August 2026.

[9] OWASP Foundation, "OWASP Top 10 - Web Application Security Risks & Injection Prevention", https://owasp.org/www-project-top-ten/, Accessed: August 2026.
[10] PHP Documentation Group, "PHP Data Objects (PDO) Manual & Prepared Statements", https://www.php.net/manual/en/book.pdo.php, Accessed: August 2026.
[11] World Wide Web Consortium (W3C), "HTML5 and CSS3 Web Standards Specification", https://www.w3.org/standards/, Accessed: August 2026.

[12] Internet Engineering Task Force (IETF), "RFC 5321: Simple Mail Transfer Protocol (SMTP)", https://datatracker.ietf.org/doc/html/rfc5321, Accessed: August 2026.